Pseudocódigo del Proceso `PAGOS_ANULACION`

Exclusión Mutua con Prioridades y Testigo

Sistemas Distribuidos

Variables y semáforos relevantes en memoria compartida

Variable	Descripción
`testigo`	El nodo posee el token (testigo)
`dentro`	Hay un proceso en la SCEM
`turno_PA`	Turno del proceso Pagos/Anulación
`turno_RA`	Turno de Reservas/Administración
`turno_C`	Turno de Consultas
`turno`	Variable de turno general
`contador_anul_pagos_pendientes`	Procesos PA esperando SCEM
`contador_procesos_max_SC`	Procesos que han entrado a SC consecutivamente
`prioridad_maxima`	Prioridad más alta en este nodo
`prioridad_max_otro_nodo`	Prioridad más alta en el otro nodo
`EVITAR_RETENCION_EM`	Umbral para forzar ceder el testigo

Región protegida por semáforos

Semáforos de la región: **sem_wait**(*sem_contador_anul_pagos_pendientes*), **sem_wait**(*sem_testigo*), **sem_wait**(*sem_turno_PA*), **sem_wait**(*sem_turno*), **sem_wait**(*sem_contador_procesos_max_SC*), **sem_wait**(*sem_dentro*), **sem_wait**(*sem_prioridad_maxima*), **sem_wait**(*sem_prioridad_max_otro_nodo*)

Pseudocódigo 1 Proceso `PAGOS_ANULACION(mi_id)`

Conectar a memoria compartida ← `shmget(mi_id)` + `shmat(...)`
timeInicio ← `gettimeofday()`

Región protegida por semáforos

sem_wait(*sem_contador_anul_pagos_pendientes*)
contador_anul_pagos_pendientes += 1
sem_wait(*sem_testigo*) · **sem_wait**(*sem_turno_PA*) · **sem_wait**(*sem_turno*) · **sem_wait**(*sem_contador_procesos_max_SC*)

si (*NOT (testigo) AND cnt* = 1) *OR (testigo AND turno_PA AND (cnt + cntSC - EVITAR_RET* = 1)) *OR (testigo AND (cnt* = 1) *AND NOT (turno_PA) AND turno*) **entonces** ▷ ¿Debo pedir el testigo?
  sem_post(*sem_testigo*) · **sem_post**(*sem_turno_PA*) · **sem_post**(*sem_contador_procesos_max_SC*) · **sem_post**(*sem_turno*) · **sem_post**(*sem_contador_anul*)
  sem_wait(*sem_turno_C*); *turno_C* ← **false**; **sem_post**(*sem_turno_C*)
  sem_wait(*sem_prioridad_maxima*); *prioridad_maxima* ← `PAGOS_ANUL`; **sem_post**(*sem_prioridad_maxima*)
  Enviar peticiones a todos los nodos: `send_peticiones(mi_id, PAGOS_ANUL)`
  sem_wait(*sem_anul_pagos_pend*) ▷ Bloquear hasta recibir el testigo
si no
  sem_post(*sem_testigo*) · **sem_post**(*sem_turno_PA*) · **sem_post**(*sem_turno*) · **sem_post**(*sem_contador_procesos_max_SC*) · **sem_post**(*sem_contador_anul*)
  sem_wait(*sem_testigo*); **sem_wait**(*sem_dentro*)
  si *dentro OR NOT (testigo)* **entonces** ▷ ¿Hay alguien dentro o no tengo testigo?
    sem_post(*sem_dentro*); **sem_post**(*sem_testigo*)
    sem_wait(*sem_anul_pagos_pend*) ▷ Esperar permiso
  si no
    sem_post(*sem_dentro*); **sem_post**(*sem_testigo*)
    sem_wait(*sem_prioridad_maxima*); *prioridad_maxima* ← `PAGOS_ANUL`; **sem_post**(*sem_prioridad_maxima*)
    sem_wait(*sem_turno_PA*); *turno_PA* ← **true**; **sem_post**(*sem_turno_PA*)
    sem_wait(*sem_turno*); *turno* ← **true**; **sem_post**(*sem_turno*)
  fin si
fin si

Sección Crítica de Exclusión Mutua (SCEM)

timeSC ← `gettimeofday()` ▷ Inicio SC
sem_wait(*sem_contador_anul*); *contador_anul* −= 1; **sem_post**(*sem_contador_anul*)
sem_wait(*sem_dentro*); *dentro* ← **true**; **sem_post**(*sem_dentro*)
sem_wait(*sem_contador_procesos_max_SC*); *cntSC* += 1; **sem_post**(*sem_contador_procesos_max_SC*)
`sleep(SLEEP)` ▷ Ejecutar tarea en la sección crítica
timeFinSC ← `gettimeofday()` ▷ Fin SC

`set_prioridad_max(me)` ▷ Recalcular prioridad máxima local

Región protegida por semáforos

sem_wait(*sem_prioridad_max_otro_nodo*); **sem_wait**(*sem_prioridad_maxima*)

si *prioridad_max_otro_nodo* > *prioridad_maxima* **entonces** ▷ Otro nodo tiene mayor prioridad